📘 TP5-II – Practice Exercises (ClassicModels)

- Teamwork duration: 2h
- Database: `classicmodels`
- Objective: Practice SQL queries, functions, stored procedures, and control flow.

🔹 Part A: SQL Basics (Warm-up)

Exercise 1: Simple SELECT

Display:

- `customerName`
- `country`
- `creditLimit`

Only customers from **USA**.

Exercise 2: ORDER BY

List all **products**:

- `productName`
- `buyPrice`
- `MSRP`

Sorted by **buyPrice DESC**.

Exercise 3: WHERE + BETWEEN

Show all **payments** where:

- amount is between **10,000 and 50,000**
- sorted by `paymentDate`

🔹 Part B: JOIN Practice

Exercise 4: Customer & Orders

Display:

- `customerName`
- `orderNumber`
- `orderDate`
- `status`

Using **customers + orders** tables.

Exercise 5: Orders & Products

Show:

- `orderNumber`
- `productName`
- `quantityOrdered`
- `priceEach`

Using **orders + orderdetails + products**.

Exercise 6: Employee & Office

Display:

- employee full name
- jobTitle
- office city
- office country

🔹 Part C: SQL Functions (Student Creates Function)

Exercise 7: Function – Customer Credit Level

Create a function `creditLevel(creditLimit DOUBLE)` Rules:

- creditLimit < 50,000 → **Low**
- 50,000–100,000 → **Medium**
- > 100,000 → **High**

Example:

```
SELECT customerName, creditLevel(creditLimit) FROM customers;
```

Exercise 8: Function – Total Order Amount

Create a function `totalOrderAmount(orderNo INT)` Return the **total price** of one order.

Hint:

```
SUM(quantityOrdered * priceEach)
```

🔹 Part D: Stored Procedures

Exercise 9: Customers by Country

Create procedure:

```
CALL getCustomersByCountry('France');
```

Return:

- customerName
- phone
- city

Exercise 10: Products by Price Range

Create procedure:

```
CALL getProductsByPrice(50, 100);
```

Return:

- productName
- buyPrice
- MSRP Sorted by buyPrice ASC.

Exercise 11: Orders by Status

Create procedure:

```
CALL getOrdersByStatus('Shipped');
```

Return:

- orderNumber
- orderDate
- customerName

🔹 Part E: Logic & Loop Practice

Exercise 12: Prime Number Function

Create function `isPrime(num INT)` Return:

- `"X is prime"`
- `"X is not prime"`

Test:

```
SELECT isPrime(11);
```

Exercise 13: Count Customer Orders

Create function `countOrders(customerNo INT)` Return number of orders per customer.

🔹 Part F: Advanced (Optional / Bonus)

Exercise 14: Top Customers

Create procedure:

```
CALL topCustomers(5);
```

Return **top 5 customers** by total payment amount.

Exercise 15: Drop Table Procedure

Create procedure:

```
CALL dropTableName('table_name');
```

Uses **dynamic SQL** safely.